GREENFINANCE-SCORER

# Technologies et outils utilisés

Chapitre pour le rapport de projet de fin d’études

| | |
|---|---|
| **PÉRIMÈTRE** | Code, dépendances, configurations et livrables du projet |
| **ÉTAT EXAMINÉ** | 12 septembre 2026 · Version applicative déclarée 0.1.0 |
| **CONTENU** | 10 sections · Inventaire par couche · Versions · Références internes |

Un inventaire fondé sur l’implémentation observée, avec une distinction explicite entre les fonctions disponibles, les configurations de déploiement et les éléments encore non implémentés.

# Sommaire

Introduction et périmètre de l’étude 3

1. Frontend 4

2. Backend 6

3. Base de données 8

4. Intelligence artificielle et traitement des données 10

5. Sécurité 13

6. Architecture et communication 15

7. Déploiement et conteneurisation 16

8. Développement et gestion du projet 17

9. Diagrammes et conception 19

10. Synthèse de la stack technologique 20

Références internes et traçabilité 24

Les versions verrouillées sont distinguées des étiquettes d’images et des versions observées sur le poste. Les références [R01] à [R24] renvoient aux fichiers du projet.

# Introduction et périmètre de l’étude

GreenFinance-Scorer est une plateforme de gestion de rapports ESG, d’extraction documentaire et d’aide à l’analyse, organisée autour de six acteurs : administrateur, entreprise, auditeur, investisseur, chercheur et institution. Son architecture associe une interface web React, une API Python, une base relationnelle PostgreSQL et une chaîne d’extraction assistée par intelligence artificielle.

Ce chapitre décrit l’état effectivement présent dans le répertoire du projet, y compris les modifications locales au moment de l’examen. L’inventaire repose sur les fichiers de dépendances, leurs versions verrouillées, les imports, les fonctions appelées, les routes, les migrations et les configurations de construction et de validation. Les documents de conception ont été confrontés au code : une dépendance déclarée, un commentaire prospectif ou un module vide ne suffisent pas à établir une fonctionnalité opérationnelle. Les références [R01] à [R24], regroupées après le tableau de synthèse, rendent les constats vérifiables.

Les justifications exposées correspondent au rôle observable de chaque technologie et à son adéquation aux besoins de la plateforme. Elles ne prétendent pas reconstituer des décisions historiques qui ne seraient pas documentées. Les versions de bibliothèques proviennent de `uv.lock` et de `frontend/package-lock.json` ; elles décrivent la résolution des dépendances, sans attester la version de chaque processus éventuellement déployé. Les versions des outils locaux et les étiquettes d’images Docker sont identifiées séparément. [R01, R18]

Le périmètre livré comprend l’extraction d’indicateurs, leur rattachement aux pièces justificatives, le circuit de contrôle humain, le scoring ESG et les espaces métier. En revanche, les modules dédiés aux facteurs d’émission, aux émissions financées PCAF et à l’explicabilité avancée ne contiennent pas encore d’implémentation. Ces limites sont précisées dans leurs rubriques afin que le chapitre reste conforme au logiciel examiné.

# 1. Frontend

## 1.1. Bibliothèque d'interface et langage

**React et React DOM** constituent le socle de l'interface. React permet de composer les écrans à partir de composants réutilisables : tableaux de bord, fiches d'entreprise, rapports, formulaires et éléments communs de navigation. React DOM assure leur rendu dans le navigateur. La séparation par fonctionnalités, sous `frontend/src/features/`, correspond aux espaces des différents acteurs. Elle facilite l'évolution d'un espace tout en conservant des composants partagés pour les profils, les indicateurs ESG et les contrôles d'interface. [R02]

**TypeScript** est le langage des fichiers `.ts` et `.tsx`. Il décrit notamment les propriétés des composants, les paramètres des requêtes et les réponses de l'API. Son intérêt est de rendre explicites les contrats entre les couches et de détecter des incompatibilités lors de la compilation. Les fichiers de configuration examinés n'activent pas globalement l'option `strict` : le typage est bien utilisé, mais le projet ne doit pas être présenté comme une application intégralement configurée en mode strict. [R01, R02]

**Vite**, avec `@vitejs/plugin-react`, fournit le serveur de développement et la construction des ressources statiques. Il prend en charge l'intégration React et le proxy local vers l'API. Le script de construction exécute d'abord TypeScript, puis Vite. Cette organisation convient à une application monopage dont les écrans communiquent avec une API distincte. Node.js intervient dans cet outillage ; le code frontend livré au navigateur est constitué de ressources HTML, CSS et JavaScript. [R01, R03]

## 1.2. Styling et composants d'interface

**Tailwind CSS**, intégré à Vite par `@tailwindcss/vite`, est utilisé dans les classes des composants et la feuille globale. Il permet de définir directement l'espacement, la mise en page, les couleurs et les états visuels. Les variables CSS et les styles communs assurent une présentation cohérente des espaces métier, y compris les thèmes clair et sombre. [R03]

Les composants locaux de **type shadcn/ui**, présents dans `shared/ui/` et décrits par `components.json`, reposent notamment sur les primitives du paquet **Radix UI**. Les boîtes de dialogue, libellés, sélecteurs et autres contrôles sont ainsi composés et adaptés dans le code du projet. Ce choix apporte des comportements d'interaction réutilisables et permet de maîtriser l'apparence des composants sans dépendre d'un thème externe imposé. shadcn/ui est ici un ensemble de sources locales ; aucune version de paquet d'exécution autonome n'est à lui attribuer. [R03]

**class-variance-authority** définit les variantes de composants, par exemple les styles de boutons et de badges. **clsx** assemble conditionnellement les classes, tandis que **tailwind-merge** résout les conflits entre utilitaires Tailwind. Leur association simplifie les composants partagés et évite de répéter la logique de présentation. **Lucide React** fournit les icônes de navigation et d'action, pour une iconographie homogène sur l'ensemble de la plateforme. [R03]

La police **IBM Plex Sans** est chargée par une importation CSS depuis **Google Fonts**. Ce service fournit la typographie web utilisée par la charte de l'interface ; la version des fichiers de police n'est pas figée dans le dépôt. [R03]

### 1.3. Gestion de l'état et routage

**TanStack Query**, distribué par `@tanstack/react-query`, gère les données issues du serveur : chargement, cache, erreurs, mutations et invalidation après modification. Il est notamment utilisé pour l'utilisateur connecté, les listes métier, les tableaux de bord et les notifications. Cette organisation maintient la cohérence entre les actions effectuées et les informations affichées, tout en évitant de réécrire un mécanisme de synchronisation dans chaque écran. [R02, R04]

L'état local utilise les mécanismes natifs de **React**, dont les hooks et le contexte. Le contexte de thème centralise la préférence d'affichage, qui peut être conservée dans le stockage local du navigateur. Les filtres, dialogues et saisies temporaires restent proches des composants concernés. Aucun gestionnaire global Zustand n'est appelé par le code frontend examiné. [R02, R03]

**React Router DOM** organise la navigation entre la connexion et les espaces protégés. Le composant `RequireRole` examine la session connue du frontend et le rôle de l'utilisateur avant de rendre une route. Le routage assure ainsi une navigation cohérente avec les responsabilités de chaque acteur ; les décisions d'autorisation restent également contrôlées par le backend. [R02, R04]

### 1.4. Graphiques, données et preuves documentaires

Les visualisations effectivement utilisées sont développées avec les capacités natives du navigateur : **SVG** pour les graphiques de comparaison et **HTML/CSS** pour les barres de répartition sectorielle des investissements. Les tableaux et cartes affichent les scores, les montants, les statuts et les indicateurs reçus de l'API. L'usage de ces éléments natifs permet de réaliser les représentations nécessaires sans bibliothèque graphique supplémentaire. [R05]

Les rapports et les preuves PDF sont affichés au moyen d'une **iframe native** et des fonctions de consultation du navigateur. Les téléchargements utilisent les API web de fichiers et de blobs. Les utilitaires **Intl** assurent le formatage des nombres, dates et devises. Les paquets Tremor et Zustand sont déclarés, mais aucun import effectif ne justifie leur inclusion dans la stack utilisée. Les bibliothèques react-pdf, visx et TanStack Table citées dans des documents de conception ne sont pas utilisées par les écrans finaux examinés. [R04, R05]

### 1.5. Formulaires, authentification et client d'API

**React Hook Form** structure les formulaires, leur état de saisie et la remontée des erreurs. **Zod** définit les schémas de validation côté navigateur. **@hookform/resolvers** relie ces schémas aux formulaires. Cette combinaison améliore la qualité des saisies et fournit un retour rapide à l'utilisateur, tout en conservant la validation serveur comme contrôle faisant autorité. [R04]

Le client réseau repose sur **Fetch**, via un adaptateur partagé. Il envoie les cookies avec les requêtes, transforme les réponses et ajoute l'en-tête CSRF aux opérations concernées. Les appels TypeScript sont générés à l'aide d'**Orval** à partir du contrat OpenAPI exporté par FastAPI. Cette génération réduit les divergences entre les routes backend et leur consommation par les composants. [R04, R20]

L'authentification frontend utilise les routes de connexion, de lecture de session et de déconnexion de l'API. Le JWT est placé dans un cookie inaccessible au JavaScript ; il n'est pas conservé comme jeton d'accès dans `localStorage`. TanStack Query maintient l'état de session, et `RequireRole` adapte l'accès aux pages et la navigation. [R04, R09]

# 2. Backend

## 2.1. Framework, langage et architecture de l'API

Le backend est développé en **Python**, avec une contrainte de projet `>=3.11` et une image Docker Python 3.11. Ce langage réunit, dans un même environnement, la logique métier, les accès aux données et les bibliothèques de traitement documentaire. [R01, R18]

**FastAPI** expose les routes HTTP et orchestre l'injection des sessions de base de données, de l'utilisateur connecté et des permissions. **Uvicorn** exécute l'application selon l'interface ASGI. **Starlette**, dépendance de FastAPI également importée directement pour certains middlewares, fournit des composants HTTP et de gestion des requêtes. Cet ensemble permet d'articuler les contrats d'API, la validation et les contrôles d'accès au niveau des routes. [R06]

L'architecture correspond à un **monolithe modulaire** : les espaces métier et les traitements spécialisés appartiennent à une même application backend, répartie en modules Python. Une sous-application FastAPI est montée sous `/api/v1`. Elle possède son propre document OpenAPI et sa documentation interactive ; la route technique `/health` reste portée par l'application racine. Les échanges suivent une organisation REST, avec des ressources accessibles par les méthodes HTTP adaptées. [R06]

## 2.2. Authentification, autorisation et rôles

**PyJWT** crée et vérifie les jetons de session, tandis que **Passlib** et **bcrypt** assurent le hachage et la vérification des mots de passe. Les routes utilisent des dépendances communes pour vérifier la validité de la session, l'activation du compte et les contraintes de changement de mot de passe. Cette centralisation évite de dupliquer les règles d'authentification dans chaque espace. [R09]

La gestion des rôles repose sur l'énumération `Role` et la dépendance `require_role`. Les six rôles sont administrateur, entreprise, auditeur, investisseur, chercheur et institution. Des contrôles métier complètent le rôle : propriété d'un rapport ou d'un portefeuille, affectation à un auditeur, rattachement d'un chercheur ou appartenance d'un projet à une institution. Cette combinaison permet de contrôler aussi bien le type d'action que la ressource précise sur laquelle elle porte. [R07, R10]

## 2.3. Validation et services métier

**Pydantic** définit les schémas d'entrée et de sortie. Les validateurs contrôlent les valeurs, les formats, les bornes et certaines cohérences entre champs. **email-validator**, utilisé par les types d'adresse électronique de Pydantic, participe au contrôle des comptes. **pydantic-settings** charge les paramètres d'environnement et les valide au démarrage. Les modèles de transport sont distincts des entités de persistance, ce qui limite l'exposition accidentelle de champs internes. [R06, R07]

Les services métier sont des fonctions Python organisées par domaine : dépôt et correction des rapports, affectation des audits, émission des avis, validation administrative, publication des entreprises, portefeuilles, projets institutionnels et analyses des chercheurs. Les notifications sont enregistrées dans la base et rendues accessibles à leur destinataire. Aucun service externe de messagerie n'est requis par ce mécanisme de notification interne. [R07, R08]

Les portefeuilles utilisent des agrégations pondérées des scores disponibles. La conversion des montants repose sur **PyYAML** et le fichier `config/fx/rates.yaml`, dont les taux sont statiques. Le taux appliqué et le montant converti sont conservés avec la position ; aucune API de change en temps réel n'est appelée. Ce fonctionnement rend les montants explicables à partir des paramètres enregistrés. [R16]

## 2.4. Traitement des rapports et gestion des fichiers

Le dépôt de rapports utilise les fonctions `UploadFile` et `Form` de FastAPI, avec **python-multipart** pour le décodage des envois `multipart/form-data`. **PyMuPDF** permet de contrôler l'ouverture du PDF, de rejeter certains documents invalides et de produire des preuves documentaires. Un checksum **SHA-256**, calculé avec la bibliothèque standard `hashlib`, participe à la détection des dépôts identiques. [R11]

Les documents sont enregistrés sur un **système de fichiers local**, au moyen de `pathlib`, sous le répertoire configuré par `STORAGE_PATH`. Les corrections sont représentées par de nouvelles versions de rapport liées au document précédent. Le code ne comporte pas d'implémentation active de stockage objet distant. Les petits avatars sont, quant à eux, contrôlés puis encodés en data URI à l'aide de `base64`, et stockés avec le profil. [R11]

Le lancement de l'extraction repose sur **BackgroundTasks de FastAPI** après l'enregistrement du rapport. Le traitement continue dans le processus applicatif avec son propre accès à la base. Il s'agit d'un traitement en arrière-plan local au serveur : aucune file distribuée Celery ou RQ, ni aucun worker séparé, n'est configuré. Redis n'est pas utilisé comme broker d'extraction. [R11, R12]

## 2.5. Journalisation et composants transversaux

**structlog** produit les journaux techniques, avec une sortie structurée en JSON en mode production et une présentation adaptée au développement. Un identifiant de corrélation relie les requêtes, les erreurs et les traces. Les gestionnaires d'exceptions normalisent les erreurs retournées et conservent les détails techniques côté serveur. Le journal d'audit métier constitue une entité distincte, persistée dans PostgreSQL, afin de retracer les actions des utilisateurs. [R06, R08]

# 3. Base de données

## 3.1. Système relationnel et connexion

**PostgreSQL** est le système de gestion de base de données configuré. Docker Compose utilise l'image `pgvector/pgvector:pg16`, ce qui établit le choix de la branche PostgreSQL 16 sans figer sa version corrective. La base conserve les comptes, les objets métier, les données extraites, les décisions d'audit et les scores. Le modèle relationnel répond au besoin de relier les résultats aux documents, aux utilisateurs et aux actions qui les ont produits. [R18]

**SQLModel** assure la définition des entités et des sessions de persistance. Il s'appuie sur **SQLAlchemy**, également utilisé directement pour les contraintes, les types SQL et certaines requêtes. **Psycopg 3**, avec sa distribution binaire déclarée, est le pilote de connexion à PostgreSQL. Les sessions sont fournies aux routes par injection de dépendances et délimitent les échanges avec la base. [R07]

Bien que l'image PostgreSQL soit issue du projet pgvector, aucune colonne vectorielle ni activation d'extension pgvector n'est définie dans les migrations examinées. La recherche sémantique réellement appelée utilise FAISS en mémoire. Il faut donc distinguer le fournisseur de l'image conteneur de l'utilisation effective d'une extension SQL vectorielle. [R07, R12, R18]

## 3.2. Modélisation et principales entités

La modélisation emploie des identifiants UUID, des clés étrangères, des champs temporels et des énumérations métier. Les modèles SQLModel décrivent les objets persistés ; les schémas Pydantic définissent les données échangées par l'API. Les principales familles d'entités sont les suivantes. [R07, R08]

| Domaine | Entités actives | Rôle dans la plateforme |
|---|---|---|
| Identité et acteurs | Utilisateur, Entreprise, InstitutionProfil, ChercheurInstitution | Porter l'identité, le rôle, le profil institutionnel et les rattachements des chercheurs. |
| Rapports et extraction | RapportESG, PreuveDocumentaire, IndicateurESG, DonneeCarbone | Conserver les versions déposées, les preuves PDF et les valeurs extraites. |
| Contrôle | AvisAudit | Formaliser l'examen et la recommandation de l'auditeur avant décision administrative. |
| Notation | ScoreESG, ConfigurationPonderation | Relier les notes au rapport et à la version de configuration utilisée. |
| Investissement | Portefeuille, PositionPortefeuille | Décrire les placements, les montants, les devises et l'agrégation des scores. |
| Recherche | Projet, AffectationProjet, Analyse, AnalyseEntreprise | Organiser les projets, leurs chercheurs affectés et les analyses d'entreprises. |
| Traçabilité et communication | JournalAudit, Notification | Conserver les événements métier et informer leurs destinataires. |

Une entité `SignalementEcart` figure également dans le schéma, mais aucun service actif de gestion de signalements n'a été identifié. Elle n'est donc pas présentée comme une fonctionnalité livrée.

### 3.3. Relations et intégrité

Les relations reposent sur les clés étrangères et les jointures SQLModel/SQLAlchemy. Une entreprise possède des rapports ; un rapport regroupe des indicateurs et des données carbone ; un score référence son rapport et sa configuration ; un portefeuille possède des positions liées à des entreprises. Les tables d'association représentent notamment le rattachement des chercheurs et le lien entre analyses et entreprises. Le chaînage des versions de rapports est assuré par une référence au rapport précédent. [R07, R08]

Les contraintes effectivement définies couvrent, selon les tables, les bornes des scores, la non-négativité de certaines valeurs, les montants et les unicités métier. Des validations applicatives complètent ces contraintes. Les énumérations sont représentées par des types SQL non natifs configurés dans le projet ; il ne faut pas en déduire que chaque colonne d'énumération dispose automatiquement d'une contrainte CHECK explicite. [R07, R17]

### 3.4. Migrations et initialisation

**Alembic** versionne l'évolution du schéma. Le répertoire examiné contient seize révisions, depuis une première révision vide et la création consolidée des tables jusqu'aux ajouts récents de données d'entreprise et d'avatar utilisateur. La commande `alembic upgrade head`, appelée par le script de démarrage local, applique les évolutions dans l'ordre des dépendances. Cette méthode conserve l'historique des changements de structure. [R17]

L'image PostgreSQL initialise la base et son utilisateur à partir des variables de configuration. La création des tables relève ensuite d'Alembic. Les tests d'intégration appliquent également les migrations ; ils utilisent par défaut une base dérivée portant le suffixe `_test`, avec possibilité de définir explicitement `TEST_DATABASE_URL`. Aucun script autonome de création du premier administrateur de production n'a été identifié ; la création ultérieure des comptes est assurée par les services d'administration. [R17, R19]

# 4. Intelligence artificielle et traitement des données

## 4.1. Extraction et structuration des rapports

**Docling** convertit les PDF en une représentation structurée comprenant le texte, les tableaux et les références de provenance. Le pipeline configure la reconnaissance de la structure des tableaux et prépare les éléments nécessaires à la recherche d'indicateurs. **docling-core**, dont certains types sont importés directement, fournit les structures de document exploitées par l'extracteur. Le découpage en fragments est réalisé par une fonction Python du projet. Les composants **docling-parse** et **docling-ibm-models** appartiennent aux dépendances internes de cette chaîne. [R12]

La reconnaissance optique est configurée avec **RapidOCR** et son moteur **PaddlePaddle**. Les modèles OCR relèvent de l'écosystème PaddleOCR, mais le chemin d'exécution vérifié passe par Docling, RapidOCR puis l'inférence PaddlePaddle. Le paquet `paddleocr` est déclaré dans les dépendances sans import direct démontré dans cette chaîne ; il ne doit pas être présenté comme l'API appelée directement par l'application. Cette précision distingue l'origine des modèles du composant logiciel réellement invoqué. [R01, R12]

**PyMuPDF** intervient dans l'inspection des PDF et la génération des pièces justificatives. Cette combinaison répond à deux besoins complémentaires : exploiter le contenu documentaire et conserver un lien vers les pages consultables par l'auditeur. [R11, R13]

## 4.2. Recherche sémantique et modèles d'embeddings

Les fragments documentaires sont vectorisés avec **BGE-M3**, chargé par la bibliothèque **FlagEmbedding**. Le modèle transforme les textes et les requêtes d'indicateurs en représentations numériques permettant une recherche par proximité sémantique. **NumPy**, importé directement, prépare les matrices de vecteurs. **FAISS CPU** construit un index `IndexFlatIP` en mémoire et retrouve les passages pertinents à transmettre à l'extraction structurée. Ce mécanisme limite le contexte soumis au modèle génératif aux éléments utiles du rapport. [R12]

**PyTorch** constitue un moteur d'exécution des modèles, et **TorchVision** fait partie de la chaîne documentaire et visuelle installée. Les sources uv sélectionnent leurs distributions CPU pour les environnements concernés ; un GPU n'est pas requis par la configuration retenue. Les bibliothèques Transformers et Sentence Transformers sont présentes comme dépendances internes de l'écosystème de modèles, sans orchestration applicative directe distincte. [R01, R12]

L'index FAISS n'est pas persisté comme base vectorielle. Le module `semantic_search.py` évoquant pgvector ne contient qu'une description de fonctionnalité future. Par ailleurs, le nom BGE-M3 identifie le modèle, mais aucune révision immuable de ses poids n'est fixée dans le code ; le verrouillage des paquets Python ne suffit donc pas à figer tous les artefacts de modèle. [R12]

## 4.3. API générative et extraction des indicateurs ESG

Le fournisseur d'IA réellement intégré est **Google Gemini**, appelé par le SDK **google-genai**. L'identifiant du modèle inscrit dans l'extracteur est `gemini-3.6-flash`. Il s'agit de la valeur configurée dans le code, et non d'une affirmation de disponibilité commerciale vérifiée auprès du fournisseur pendant cet audit. Le SDK transmet les passages sélectionnés et force un appel de fonction dont les arguments structurés sont validés par les schémas Pydantic de l'application. Aucun appel actif à Anthropic/Claude ni à un service OpenAI n'a été identifié dans le backend final. [R12, R13]

L'extracteur cible les émissions Scope 1, Scope 2 selon les méthodes market-based et location-based, Scope 3, plusieurs intensités carbone, ainsi qu'un socle social et de gouvernance : part de femmes dans le management, décès professionnels et part de femmes au conseil. Les informations extraites sont associées à une période, une

unité, une page ou une preuve selon le type d'objet enregistré. Cette structuration transforme les passages de rapports en données utilisables par le contrôle humain et le scoring. [R12, R13]

Le projet comporte un **mode de démonstration** : si la clé Gemini correspond au marqueur d'exemple reconnu, l'extracteur génère des valeurs synthétiques et des preuves signalées `[DEMO]`. Ce résultat ne constitue pas une extraction IA réelle. La configuration locale examinée sélectionne le chemin Gemini et non ce marqueur de démonstration ; aucun appel distant n'a toutefois été exécuté pour la rédaction du chapitre. Le mode production refuse les marqueurs d'exemple connus au démarrage. [R06, R12]

## 4.4. Calcul des scores ESG

Le score ESG est calculé par un **moteur déterministe Python**, distinct du modèle génératif. **PyYAML** lit `config/weights/default.yaml` et **Pydantic** valide la configuration. Chaque indicateur est normalisé sur l'échelle 0–100 à partir de bornes et d'un sens de préférence, puis agrégé dans son pilier. Les notes des piliers sont à leur tour pondérées pour obtenir le score global. [R14]

Pour un indicateur de valeur x, de borne basse a et de borne haute b, la fraction normalisée vaut :

**f = min(1, max(0, (x − a) / (b − a))).**

Si une valeur élevée est favorable, la sous-note est `100 × f` ; dans le cas inverse, elle est `100 × (1 − f)`. Le score d'un pilier est la moyenne pondérée des sous-notes disponibles. Le score global est la moyenne pondérée des piliers calculables. La configuration de référence version 1 affecte un poids de 0,50 à l'environnement et de 0,25 à chacun des piliers social et gouvernance. [R14]

Un indicateur absent n'est pas assimilé à zéro : il est exclu de l'agrégation et les poids restants sont renormalisés. Un pilier sans indicateur exploitable demeure nul au sens de l'absence de valeur (`NULL`). Si aucun pilier n'est calculable, le service refuse de créer un score. Le calcul est appelé lors de la validation administrative du rapport et le résultat référence une configuration versionnée. Le code prévoit un modèle de configuration, mais le moteur utilise effectivement la méthodologie de référence fournie par fichier YAML. [R14]

Les bornes du fichier de référence sont des choix internes de première version ; le commentaire de configuration indique qu'elles ne constituent pas un étalonnage externe par secteur. Le score représente donc la méthodologie propre à la plateforme, sans revendication de certification ou de notation réglementaire. [R14]

## 4.5. Scope 1, Scope 2, Scope 3 et empreinte carbone

Dans la version examinée, les émissions sont **extraites des rapports publiés par les entreprises**. Le pipeline conserve notamment les deux variantes du Scope 2 et les intensités explicitement présentes dans les documents. L'interface restitue ces valeurs et leurs métadonnées ; elle ne reconstitue pas un bilan d'émissions à partir de consommations d'énergie, de déplacements, d'achats ou de facteurs d'émission. [R05, R12, R13]

La valeur extraite est copiée dans un champ exprimé en tonnes de CO2 équivalent, sans moteur indépendant de conversion des unités dans cette étape. La cohérence de l'unité interprétée demeure donc un point à vérifier lors de l'examen des preuves ; la validation de la structure JSON ne suffit pas à établir l'exactitude physique de la mesure. [R12, R13]

Les fichiers `app/carbon/emission_factors.py` et `app/carbon/pcaf.py` ne contiennent pas de fonctions de calcul. Le chemin `EMISSION_FACTORS_PATH` est déclaré, mais aucun moteur actif de chargement et d'application des facteurs n'a été identifié. Il n'existe donc pas de technologie opérationnelle de calcul d'empreinte carbone indépendante ni de calcul des émissions financées PCAF à attribuer à cette version. Les agrégations de portefeuille concernent les scores ESG et leur couverture, pas les émissions financées. [R15, R16]

La valeur `score_qualite_pcaf` affectée par l'extracteur est fixée à 3 et commentée comme provisoire. Elle ne résulte pas d'un moteur d'évaluation de qualité PCAF ; sa présence dans une entité ou une réponse d'API ne suffit pas à démontrer une implémentation de cette méthodologie. [R12, R15]

## 4.6. Explicabilité effectivement disponible

L'explicabilité repose sur la **traçabilité documentaire** et la **transparence du calcul** : document d'origine, page de référence, extrait, unité, période, preuve PDF, avis d'audit, notes E/S/G et référence de configuration. **PyMuPDF** génère les documents de preuve, qui sont présentés avec les indicateurs dans le frontend. La décomposition par pilier et la disponibilité des règles YAML permettent de comprendre la construction du score. [R05, R13, R14]

Les modules `explainability/decomposition.py` et `explainability/justification.py` sont encore descriptifs. Aucune bibliothèque SHAP ou LIME, ni aucun service autonome de justification avancée, n'est appelé. La complétude dispose également d'un module descriptif non implémenté ; la couverture calculée pour les portefeuilles ne doit pas être confondue avec un moteur général de contrôle de complétude documentaire. [R15, R16]

# 5. Sécurité

## 5.1. Authentification et gestion des sessions

Les mots de passe sont hachés avec **bcrypt**, par l'intermédiaire de **Passlib**. Le projet borne bcrypt à une version inférieure à 4.1 pour assurer la compatibilité avec Passlib 1.7.4 ; la version verrouillée est 4.0.1. Ce mécanisme conserve une empreinte de vérification plutôt qu'un mot de passe en clair. [R01, R09]

Les jetons **JWT**, signés avec **HS256** par PyJWT, contiennent l'identité, le rôle, une génération de session (`gen`), la date d'émission et la date d'expiration. Leur durée de validité configurée dans le code est de douze heures. La session est transmise par un cookie `HttpOnly`, `SameSite=Lax`, nommé `__Host-access_token`. Les attributs `Secure` et `Path=/`, sans attribut `Domain`, sont appliqués systématiquement, y compris en développement local. Le cookie CSRF suit les mêmes attributs de portée mais reste lisible par JavaScript pour construire l'en-tête attendu. [R09]

**Redis** conserve un compteur de génération par utilisateur pour la révocation des sessions. Le backend compare ce compteur au champ `gen` du jeton et contrôle l'état du compte à chaque accès protégé. La déconnexion et les opérations sensibles sur les comptes peuvent ainsi rendre les anciennes sessions inutilisables avant leur expiration naturelle. Un jeton expiré impose une nouvelle connexion ; aucun renouvellement automatique par refresh token n'est implémenté. Aucun fournisseur d'identité externe, SSO ou mécanisme MFA opérationnel n'est intégré, même si une variable de configuration relative à MFA subsiste. [R09, R10]

## 5.2. Protection des API

Le middleware **CSRF** vérifie les requêtes modifiant l'état lorsqu'une session authentifiée est présente, hors route de connexion. Le frontend lit le cookie CSRF et transmet sa valeur dans l'en-tête `X-CSRF-Token`. Le backend compare cet en-tête à la signature **HMAC-SHA256** recalculée à partir de l'utilisateur et de sa génération de session, avec une comparaison à temps constant. Le middleware contrôle également les en-têtes d'origine ou de référent lorsqu'ils sont présents. [R04, R10]

La politique **CORS** repose sur une liste explicite d'origines, compatible avec l'utilisation de cookies. La configuration rejette l'origine générique `*`. Une limitation des échecs de connexion utilise Redis : cinq échecs sur une fenêtre de quinze minutes pour une même adresse électronique. Cette mesure protège le point d'authentification ; le dépôt ne définit pas de limitation globale de débit pour toutes les routes. [R06, R10]

## 5.3. Autorisation entre les acteurs

Les rôles définissent l'accès aux familles d'API, et les services vérifient le périmètre des ressources. Les principaux contrôles sont résumés ci-dessous. [R07, R10]

| Acteur | Périmètre d'accès contrôlé |
|---|---|
| Administrateur | Gestion des comptes, affectation des audits, validation administrative et publication. |
| Entreprise | Profil de l'entreprise associée, dépôt et consultation de ses propres rapports et corrections. |
| Auditeur | Rapports affectés et avis relevant de son périmètre d'audit. |
| Investisseur | Entreprises publiées et gestion de ses propres portefeuilles et positions. |
| Chercheur | Données d'entreprises publiées, rattachements, projets affectés et analyses autorisées. |
| Institution | Projets de son institution, chercheurs rattachés et analyses relevant de ce périmètre. |

Ces règles sont appliquées côté serveur. Le masquage d'une action dans l'interface apporte une cohérence ergonomique, tandis que les dépendances et contrôles de propriété protègent effectivement les données contre un appel direct à l'API.

## 5.4. Validation et sécurisation des données

Les contrôles se répartissent entre Pydantic pour les entrées, les services pour les règles métier et les contraintes SQL pour l'intégrité persistée. Les uploads PDF sont limités à **50 Mio**, contrôlés par signature et ouverts avec PyMuPDF. Les images de profil sont limitées à **2 Mio** et restreintes par signature aux formats PNG, JPEG et WebP. Il s'agit de contrôles de type et de taille ; aucun antivirus intégré n'est identifié. [R07, R11]

Les chemins de stockage sont construits côté serveur à partir d'identifiants, et les téléchargements passent par les contrôles d'accès applicatifs. La configuration de production rejette certains secrets d'exemple. Les gestionnaires d'erreurs de l'API métier retournent des messages normalisés ; le journal d'audit trace notamment les opérations sur les comptes et les sessions. Le dépôt ne fournit pas de terminaison TLS ni de dispositif externe de gestion de secrets : ces services ne peuvent pas être listés comme composants déployés. [R06, R08, R18]

# 6. Architecture et communication

## 6.1. Organisation générale

La plateforme associe quatre ensembles : l'application web, l'API et ses modules métier, les services de persistance et le pipeline documentaire. Les modules d'ingestion et de scoring sont appelés depuis les services métier ; ils ne constituent pas des microservices déployés séparément. PostgreSQL conserve les données structurées, Redis sert aux mécanismes de sécurité et le stockage local conserve les documents. L'API Gemini constitue le service externe d'IA intégré. [R06, R12, R18]

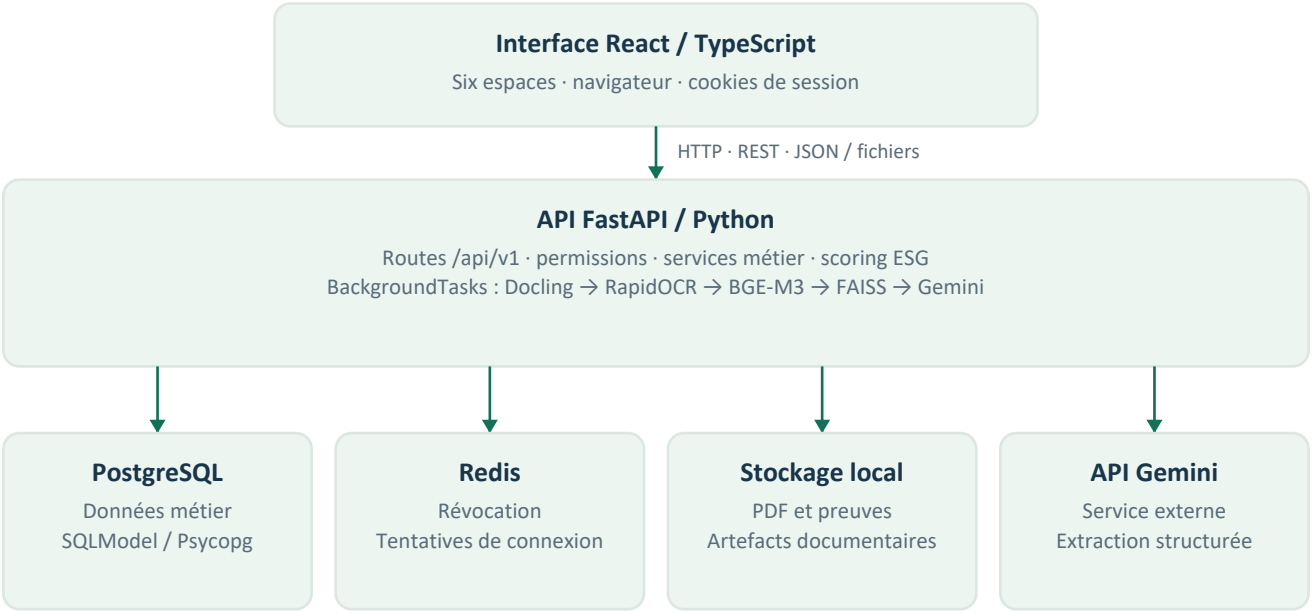



Figure 1 — Architecture logique constatée dans le code du projet.

Le schéma de synthèse représente l'organisation constatée dans le code. Il a été établi pour ce chapitre et ne suppose pas l'existence d'un outil de diagramme supplémentaire dans la stack de l'application.

## 6.2. Communication Frontend ↔ Backend

Les appels du navigateur utilisent **HTTP**, les conventions **REST** et principalement le format **JSON**. Le préfixe `/api/v1` identifie le contrat métier. Les documents et images sont envoyés en `multipart/form-data` ; les PDF et exports CSV sont retournés comme contenus de fichiers. Les dates, identifiants et énumérations sont sérialisés selon les schémas exposés par l'API. [R04, R06, R11]

En développement, le **proxy Vite** redirige `/api/v1` vers `http://localhost:8000`, depuis le frontend servi sur le port 5173. Le navigateur peut ainsi utiliser des chemins relatifs avec les cookies de session. La configuration CORS couvre les appels autorisés entre origines lorsque cette situation se présente. Aucun protocole WebSocket, GraphQL ou bus d'événements interservices n'est utilisé par les fonctionnalités examinées. [R03, R06]

## 6.3. Contrat et organisation des services

**OpenAPI** formalise les routes, paramètres et schémas. Le script `export_openapi.py` exporte le schéma de la sous-application FastAPI ; **Orval** en dérive les appels TypeScript et les types frontend. Le contrôle `api:check` régénère ces fichiers puis recherche une différence Git, afin de détecter un décalage de contrat. La documentation interactive est exposée sous `/api/v1/docs`, et le schéma sous `/api/v1/openapi.json`. [R06, R20]

# 7. Déploiement et conteneurisation

## 7.1. Docker et Docker Compose

**Docker** fournit un environnement Linux reproductible pour le backend et ses bibliothèques natives. Le `Dockerfile` utilise `python:3.11-slim`, installe les bibliothèques système nécessaires au traitement documentaire et exécute `uv sync --frozen` à partir du verrou de dépendances. Les paquets système comprennent notamment `libgl1`, `libglib2.0-0`, `libsm6`, `libxext6`, `libxrender1`, `libgomp1` et `libxcb1`. Ils satisfont les besoins d'exécution de la chaîne OCR et des composants graphiques. [R18]

**Docker Compose** déclare trois services : `api`, `db` et `redis`. L'API est exposée sur le port 8000, PostgreSQL sur 5432 et Redis sur 6379. Des contrôles de santé avec `pg_isready` et `redis-cli ping` ordonnent le démarrage des dépendances. Le volume nommé `pgdata` conserve les données PostgreSQL, tandis que le montage de `./storage` conserve les rapports et preuves de l'API. [R18]

L'image Redis est étiquetée `redis:7-alpine` et l'image de base `pgvector/pgvector:pg16`. Ces étiquettes décrivent une famille de versions ; elles ne fixent pas un digest ou une version corrective exacte. Les versions relevées des outils locaux Docker et Compose figurent dans le tableau de synthèse, sans être assimilées aux versions des services conteneurisés.

## 7.2. Configuration des environnements

Le fichier `.env.example` documente les paramètres attendus : connexions PostgreSQL et Redis, secret de session, origine frontend, clé Gemini, stockage, pondérations et taux de change. Le fichier `.env` réel est exclu du suivi Git. **pydantic-settings** lit et valide ces paramètres. Les fichiers **YAML** conservent les paramètres métier versionnés de scoring et de conversion monétaire. [R06, R14, R16, R18]

Le script **Bash** `scripts/dev-up.sh` démarre PostgreSQL et Redis avec Compose puis applique les migrations. Le README prévoit ensuite le lancement local d'Uvicorn et de Vite. Le poste examiné utilise Windows et PowerShell, tandis que la CI s'exécute dans un environnement Ubuntu. Ces outils constituent les environnements de développement et de validation observables. [R18, R19]

## 7.3. Déploiement effectivement décrit par le dépôt

Le dépôt fournit la conteneurisation de l'API et de ses services de données, ainsi qu'un processus local de lancement du frontend. **Aucun service frontend n'est déclaré dans Docker Compose**, et aucun Dockerfile frontend ni configuration de serveur web de production n'a été identifié. `npm run build` produit les ressources statiques, mais leur hébergement en production n'est pas décrit par un service livré. [R01, R18]

Le Dockerfile backend copie le code applicatif et les tests, mais pas les répertoires `config/` et `alembic/`, ni `alembic.ini`. Les réglages de scoring et de change par défaut ne sont donc pas embarqués par cette recette, et les migrations doivent être appliquées depuis l'environnement qui les possède. Le Compose contient par ailleurs une URL de base utilisant des identifiants d'exemple. Ces éléments limitent la portée de la configuration actuelle : elle ne démontre pas un déploiement de production complet et autonome. [R18]

Aucun fournisseur cloud, hébergement public, reverse proxy, orchestrateur Kubernetes ou pipeline de livraison continue n'est défini dans le dépôt. Le workflow GitHub Actions réalise l'intégration continue et la validation, sans étape de déploiement automatique. Cette distinction évite de confondre les outils disponibles pour construire et tester le projet avec un environnement d'exploitation effectivement livré. [R19]

# 8. Développement et gestion du projet

## 8.1. Éditeur, versionnement et collaboration

L'usage d'un **IDE précis ne peut pas être établi** à partir des sources disponibles. Les exclusions `.vscode/` et `.idea/` du fichier Git ne prouvent ni Visual Studio Code ni PyCharm. Le chapitre laisse donc l'éditeur non renseigné plutôt que d'attribuer au projet un outil sans preuve.

**Git** assure le suivi des modifications et l'historique du code. Le dépôt possède un remote **GitHub**, `cheikhmama/GreenFinance-Scorer`, et un workflow **GitHub Actions** déclenché sur les push et pull requests. Ces outils relient le versionnement aux contrôles automatisés. Le commit de référence au moment de l'audit est `3d59daa`, daté du 10 septembre 2026, complété par les modifications locales présentes lors de l'examen. [R19, R24]

## 8.2. Gestion des dépendances et construction

**uv** gère l'environnement Python, la résolution des paquets et leur exécution. `pyproject.toml` déclare les dépendances principales et les groupes d'outillage ; `uv.lock` fixe leur résolution. **Hatchling** est le backend de construction déclaré pour le paquet Python, ce qui formalise son assemblage sans qu'une distribution publiée soit attestée. [R01, R18]

**npm** gère les dépendances frontend, et `package-lock.json` en conserve les versions résolues. La CI utilise `npm ci` pour installer à partir du verrou. **Node.js 20** est la branche demandée dans la CI ; le poste examiné dispose de Node.js 24.11.1. Cette différence doit être indiquée comme une différence d'environnement, et non transformée en une version unique supposée du projet. [R01, R19]

Les scripts npm centralisent le lancement de Vite, la construction TypeScript, la validation Biome, les tests Vitest et la génération Orval. Les paquets `@types/react`, `@types/react-dom` et `@types/node` fournissent les déclarations de types nécessaires à la compilation de cet outillage et des composants.

## 8.3. Tests et validation

**pytest** organise les tests unitaires et d'intégration backend. Les cas existants couvrent notamment les tokens, mots de passe, permissions, paramètres, erreurs, protections CSRF/CORS, contraintes de données, routes et scoring. **HTTPX**, utilisé par le client de test FastAPI, permet d'exercer les API. Les tests d'intégration s'appuient sur PostgreSQL et Redis réels, avec application des migrations. [R19]

**pytest-cov**, fondé sur **coverage.py**, mesure la couverture du code lors de la CI. **python-dotenv** est importé par les fixtures pour lire la configuration avant d'isoler la base de test. Les fixtures peuvent substituer le pipeline d'extraction afin de tester les flux métier de façon déterministe ; l'existence de ces tests ne prouve donc pas, à elle seule, l'exécution des modèles IA distants. [R19]

Côté frontend, **Vitest**, **Testing Library React**, **Testing Library DOM**, **jest-dom** et **jsdom** permettent de vérifier les composants et les comportements utilisateur dans un environnement DOM simulé. Les tests des pages de connexion et du contrôle `RequireRole` concernent les fonctionnalités applicatives. Des tests séparés existent pour le prototype ; leurs outils exclusifs ne sont pas assimilés à la stack des écrans livrés. Playwright est déclaré comme dépendance de développement, mais aucune suite, configuration ou commande Playwright versionnée n'a été identifiée. [R21]

Le corpus `data_test/reference_e2e/` contient des rapports synthétiques et des scénarios destinés à vérifier le circuit métier. Le script `generate_reference_e2e_reports.py` produit les PDF avec PyMuPDF. Ce corpus facilite la reproduction des cas de dépôt et de contrôle, tout en restant distinct de rapports d'entreprise réels et d'une mesure de performance des modèles. [R22]

## 8.4. Qualité du code et intégration continue

**Ruff** vérifie le code Python ; **mypy**, avec le plugin Pydantic et des paquets de types pour Passlib et PyYAML, réalise l'analyse statique de types. **Biome** fournit le lint et le formatage frontend. TypeScript ajoute le contrôle de compilation, et le contrôle Orval/Git vérifie la synchronisation du client d'API. La CI exécute ces étapes, les tests backend avec couverture, puis la construction et les tests frontend. [R01, R19, R20]

Le paquet pre-commit figure dans le groupe de développement, mais aucune configuration de hooks ni invocation dédiée n'est livrée. Il ne fait donc pas partie des contrôles automatisés démontrés. Les résultats d'une nouvelle exécution complète de la CI n'ont pas été établis pendant cet audit documentaire : les outils et les contrôles sont attestés par leurs configurations et leurs tests, sans annoncer un taux de couverture ou un succès récent non mesuré.

## 8.5. Documentation et supports

**Markdown** est utilisé pour le README, les documents d'architecture, la matrice de permissions et les supports de présentation. **OpenAPI** fournit une documentation directement dérivée des routes FastAPI. Les fichiers de conception doivent être lus selon leur date et confrontés à l'implémentation, car certains conservent des choix cibles désormais différents du code. [R20, R24]

Le script `generate_final_presentation.py` utilise **python-pptx** pour produire des supports PowerPoint modifiables. Cet outil appartient à la préparation de la soutenance, sans intervenir dans l'exécution de la plateforme. Sa présence est attestée par le script et les livrables ; sa version n'est pas verrouillée dans les dépendances projet. Les notebooks et leur environnement Jupyter relèvent d'expérimentations documentaires conservées dans le dépôt : ils ne sont pas requis pour utiliser les routes applicatives et ne sont pas intégrés à la synthèse de la stack finale. [R22]

# 9. Diagrammes et conception

## 9.1. Diagramme de cas d'utilisation

Le fichier `docs/uml/diagramme-cas-utilisation-reporting-esg.drawio` contient un diagramme au format **draw.io / diagrams.net**, fondé sur une structure XML `mxGraphModel`. Il représente les acteurs et leurs interactions avec la plateforme de reporting ESG. Ce format permet de conserver le diagramme comme source éditable aux côtés du code et d'en faire évoluer la présentation. La version de l'éditeur employé n'est pas indiquée dans le fichier. [R23]

## 9.2. Classes, séquences et architecture

Les classes et relations sont effectivement décrites dans les modèles SQLModel et les migrations Alembic. Ces fichiers constituent des sources de conception exploitables, mais ne sont pas assimilés à un diagramme UML de classes déjà réalisé. De même, aucun fichier de diagramme de séquence n'a été identifié. Il n'existe pas de preuve d'usage de PlantUML ou de Mermaid pour ces livrables. [R07, R17, R23]

L'organisation de l'architecture est documentée en Markdown et représentée dans les supports de soutenance produits avec python-pptx. Les formes natives des diapositives constituent ici le mécanisme de construction des schémas de présentation. Aucun fichier Figma, outil de modélisation UML supplémentaire ou service externe de diagrammes n'est nécessaire pour expliquer les artefacts présents. [R22, R24]

| Type de conception | Support réellement identifié | Portée de la preuve |
|---|---|---|
| Cas d'utilisation | Fichier `.drawio`, format diagrams.net | Diagramme source éditable présent. |
| Classes et relations | Modèles SQLModel et migrations | Modélisation implémentée ; diagramme UML autonome absent. |
| Séquences métier | Routes, services et documentation Markdown | Enchaînements décrits dans le code ; diagramme source absent. |
| Architecture | Documents Markdown et formes python-pptx | Documentation et schémas de présentation présents. |

# 10. Synthèse de la stack technologique

Le tableau ci-dessous reprend les technologies dont un usage applicatif, un rôle d'infrastructure configuré ou un usage d'outillage effectif a été établi. **V** désigne une version résolue par un fichier de verrouillage ; **C**, une version ou une étiquette de configuration ; **L**, une version observée sur le poste d'audit ; **N/D**, une version non déterminable à partir du projet. Les entrées « source locale » désignent du code appartenant au projet. Les dépendances transitives centrales sont signalées dans leur rôle ; le tableau n'attribue pas de choix autonome à chaque sous-dépendance technique des environnements Python et npm.

## 10.1. Frontend et communication

| Technologie | Version | Rôle | Justification de son utilisation |
|---|---|---|---|
| React / React DOM | 19.2.8 / 19.2.8 (V) | Composants et rendu navigateur | Réutiliser les interfaces dans six espaces métier. |
| TypeScript | 6.0.3 (V) | Typage frontend | Expliciter les contrats et vérifier la compilation. |
| Vite / plugin React | 8.2.1 / 6.0.5 (V) | Développement et construction | Servir l'application localement et produire les ressources statiques. |
| Tailwind CSS / plugin Vite | 4.3.3 / 4.3.3 (V) | Styles et mise en page | Harmoniser la présentation et intégrer la génération CSS. |
| Composants shadcn/ui locaux | Source locale | Composants réutilisables | Maîtriser le code et l'apparence des contrôles. |
| Radix UI | 1.6.7 (V) | Primitives d'interaction | Réutiliser les comportements des contrôles d'interface. |
| class-variance-authority | 0.7.1 (V) | Variantes visuelles | Centraliser les variantes de composants. |
| clsx / tailwind-merge | 2.1.1 / 3.6.0 (V) | Composition des classes | Construire les styles conditionnels et résoudre les conflits. |
| Lucide React | 1.32.0 (V) | Icônes | Unifier les repères visuels et les actions. |
| IBM Plex Sans / Google Fonts | Police et service, N/D | Typographie web | Appliquer la charte typographique de l'interface. |
| TanStack React Query | 5.101.4 (V) | Cache et état serveur | Synchroniser les données après lecture et mutation. |
| Hooks et Context React | React 19.2.8 | État local et thème | Partager l'état nécessaire avec les mécanismes déjà disponibles. |
| React Router DOM | 7.18.2 (V) | Navigation | Organiser les routes par acteur et contrôler leur accès visuel. |
| React Hook Form | 7.85.0 (V) | Formulaires | Gérer les saisies et les erreurs de manière cohérente. |
| Zod / hookform resolvers | 4.4.3 / 5.9.1 (V) | Validation frontend | Relier schémas et formulaires. |
| SVG, HTML/CSS, iframe, Intl | API natives | Graphiques, PDF, formats | Couvrir les besoins d'affichage sans bibliothèque additionnelle. |
| Fetch, JSON, multipart, CSV | Standards/API natives | Échanges et fichiers | Communiquer avec l'API et transférer les documents. |

| Technologie | Version | Rôle | Justification de son utilisation |
|---|---|---|---|
| Orval | 8.27.0 (V) | Client d'API généré | Synchroniser les appels TypeScript avec OpenAPI. |

## 10.2. Backend, persistance et sécurité

| Technologie | Version | Rôle | Justification de son utilisation |
|---|---|---|---|
| Python | >=3.11 (C) ; 3.11.9 (L) | Langage backend | Réunir API, calcul et traitement documentaire. |
| FastAPI | 0.141.1 (V) | API REST | Relier routes, validation et injection de dépendances. |
| Uvicorn | 0.52.1 (V) | Serveur ASGI | Exécuter l'application FastAPI. |
| Starlette | 1.6.0 (V) | Composants HTTP et middlewares | Fournir les primitives de traitement des requêtes. |
| Pydantic | 2.13.4 (V) | Schémas et validation | Vérifier les entrées, sorties et données extraites. |
| pydantic-settings | 2.15.0 (V) | Configuration | Centraliser et valider les paramètres d'environnement. |
| email-validator | 2.3.0 (V) | Validation des e-mails | Contrôler les formats via les types Pydantic. |
| python-multipart | 0.0.32 (V) | Uploads | Lire les formulaires contenant des fichiers. |
| BackgroundTasks | FastAPI 0.141.1 | Extraction en arrière-plan | Déclencher le traitement après le dépôt du rapport. |
| structlog | 26.1.0 (V) | Logs techniques | Relier les événements et faciliter le diagnostic. |
| PostgreSQL | 16, image pgvector/pgvector:pg16 (C) | Base relationnelle | Conserver données métier, relations et historique. |
| SQLModel | 0.0.39 (V) | Modèles et sessions ORM | Relier les objets Python au schéma relationnel. |
| SQLAlchemy | 2.0.51 (V) | Requêtes et contraintes | Exprimer l'intégrité et les opérations SQL. |
| Psycopg / distribution binaire | 3.3.4 (V) | Pilote PostgreSQL | Assurer la connexion native à la base. |
| Alembic | 1.19.1 (V) | Migrations | Versionner et appliquer l'évolution du schéma. |
| Redis serveur / client Python | 7-alpine (C) / 8.1.0 (V) | Révocation et limitation de connexion | Partager les compteurs de tentatives et de révocation. |
| PyJWT / JWT HS256 | 2.13.0 (V) / algorithme | Sessions signées | Vérifier l'authenticité et l'expiration des jetons. |
| Passlib / bcrypt | 1.7.4 / 4.0.1 (V) | Mots de passe | Stocker et vérifier des empreintes de hachage. |
| RBAC, cookies, CSRF, CORS | Code local / standards | Contrôle d'accès | Restreindre les actions, ressources et origines autorisées. |

| Technologie | Version | Rôle | Justification de son utilisation |
|---|---|---|---|
| hashlib, hmac, secrets, pathlib, base64 | Bibliothèque standard Python | Intégrité, jetons et fichiers | Fournir les primitives nécessaires sans service externe. |

## 10.3. IA et traitement des données

| Technologie | Version | Rôle | Justification de son utilisation |
|---|---|---|---|
| Docling | 2.119.0 (V) | Conversion documentaire | Structurer textes, tableaux et provenance des PDF. |
| docling-core | 2.91.0 (V) | Modèles de document | Exploiter une représentation documentaire structurée. |
| docling-parse / docling-ibm-models | 7.12.1 / 3.14.0 (V) | Composants internes Docling | Soutenir la lecture et la compréhension de structure. |
| RapidOCR / PaddlePaddle | 3.9.2 / 3.3.1 (V) | OCR et moteur d'inférence | Reconnaître le texte des documents traités par Docling. |
| PyMuPDF | 1.28.2 (V) | Validation PDF et preuves | Contrôler les documents et produire les justificatifs. |
| FlagEmbedding / BGE-M3 | 1.4.0 (V) / poids non figés | Embeddings | Rechercher les passages proches d'une requête ESG. |
| NumPy | 2.3.5 (V) | Matrices numériques | Préparer les vecteurs exploités par FAISS. |
| FAISS CPU | 1.15.0 (V) | Index vectoriel en mémoire | Sélectionner les passages pertinents sans service vectoriel distant. |
| PyTorch / TorchVision | 2.13.0+cpu / 0.28.0+cpu (V, Windows/Linux) | Exécution des modèles | Exécuter les composants de modèles sur CPU. |
| Transformers / Sentence Transformers | 5.8.1 / 5.7.0 (V) | Dépendances internes de modèles | Fournir l'écosystème de chargement et représentation des textes. |
| google-genai / Gemini | 2.22.0 (V) / gemini-3.6-flash (C) | Extraction structurée par API | Transformer les passages documentaires en données validables. |
| PyYAML | 6.0.2 (V) | Configuration scoring et change | Séparer les paramètres métier du code. |
| Moteur ESG Python | Source locale ; méthodologie v1 | Normalisation et pondération | Produire un calcul déterministe et traçable. |

## 10.4. Déploiement, développement et conception

| Technologie | Version | Rôle | Justification de son utilisation |
|---|---|---|---|
| Docker / Docker Compose | 29.7.2 / 5.4.0 (L) | Conteneurs et services locaux | Reproduire l'environnement de l'API et des données. |
| Image Python slim | python:3.11-slim (C) | Base du conteneur API | Fournir Python et les dépendances système nécessaires. |
| Bibliothèques système Linux | Non figées dans apt | Support OCR et natif | Satisfaire les besoins des composants documentaires. |

| Technologie | Version | Rôle | Justification de son utilisation |
|---|---|---|---|
| uv | 0.11.14 (L) | Dépendances et exécution Python | Installer l'environnement à partir de uv.lock. |
| Hatchling | N/D ; backend déclaré | Construction du paquet Python | Définir l'assemblage de l'application. |
| Node.js / npm | 24.11.1 / 11.6.2 (L) ; Node 20 en CI (C) | Outillage frontend | Installer, construire et vérifier le frontend. |
| Types React / React DOM / Node | 19.2.18 / 19.2.4 / 24.13.3 (V) | Déclarations TypeScript | Typer les composants et l'environnement de construction. |
| Git / GitHub | 2.52.0.windows.1 (L) / service | Versionnement et dépôt distant | Conserver l'historique et partager le code. |
| GitHub Actions | YAML versionné ; actions v4/v3/v4 (C) | Intégration continue | Automatiser les contrôles sur push et pull request. |
| Bash / PowerShell | N/D | Scripts et terminal | Exécuter les commandes de développement. |
| Ruff / mypy | 0.16.2 / 2.3.0 (V) | Analyse statique Python | Détecter les incohérences et erreurs de code. |
| types-passlib / types-PyYAML | 1.7.7.20260211 / 6.0.12.20260906 (V) | Types pour mypy | Étendre les vérifications aux bibliothèques utilisées. |
| Biome | 2.5.9 (V) | Lint et formatage frontend | Maintenir des contrôles et conventions communs. |
| pytest / HTTPX | 9.1.1 / 0.28.1 (V) | Tests backend et HTTP | Vérifier les services, routes et permissions. |
| pytest-cov / coverage.py | 7.1.0 / 7.15.4 (V) | Couverture des tests | Mesurer le code exercé par les tests. |
| python-dotenv | 1.2.2 (V) | Configuration des fixtures | Préparer une base de test distincte. |
| Vitest / jsdom | 4.1.11 / 29.1.1 (V) | Tests frontend | Exécuter les tests avec un DOM simulé. |
| Testing Library React / DOM / jest-dom | 16.3.2 / 10.4.1 / 7.0.1 (V) | Vérification des composants | Tester les comportements et assertions DOM. |
| Markdown / OpenAPI | Formats ; version API générée | Documentation | Relier documents techniques et contrat d'API. |
| python-pptx | 1.0.2 (L), non verrouillée | Supports de soutenance | Produire des diapositives et schémas modifiables. |
| draw.io / diagrams.net | N/D ; format mxGraph | Diagramme de cas d'utilisation | Conserver un diagramme source éditable. |

# Références internes et traçabilité

Les références suivantes désignent les sources primaires du projet examinées le 12 septembre 2026. Les chemins sont relatifs à la racine du dépôt. L'audit n'a pas modifié le code applicatif, appelé les services d'IA ni exécuté une nouvelle campagne de tests. Il établit les technologies à partir de leur usage dans le code et de leur configuration. Les outils employés uniquement pour composer le présent chapitre ne sont pas ajoutés à la stack de l'application.

**[R01] Dépendances et versions :** `pyproject.toml`, `uv.lock`, `frontend/package.json`, `frontend/package-lock.json`.

**[R02] Socle frontend :** `frontend/src/main.tsx`, `frontend/src/App.tsx`, `frontend/src/shared/RequireRole.tsx`, `frontend/src/features/*/routes.tsx`, `frontend/tsconfig.app.json`, `frontend/tsconfig.node.json`.

**[R03] UI et développement frontend :** `frontend/vite.config.ts`, `frontend/components.json`, `frontend/src/index.css`, `frontend/src/shared/ui/`, `frontend/src/shared/theme/ThemeProvider.tsx`.

**[R04] Formulaires et client réseau :** `frontend/src/shared/api/client.ts`, `frontend/src/shared/api/download.ts`, `frontend/src/features/auth/api.ts`, `frontend/src/features/auth/schemas.ts`, `frontend/src/features/auth/components/LoginPage.tsx`.

**[R05] Visualisations et preuves :** `frontend/src/features/investor/components/ComparisonResultsPage.tsx`, `frontend/src/features/investor/components/InvestorDashboardPage.tsx`, `frontend/src/shared/esg/EsgSummary.tsx`, `frontend/src/shared/esg/EvidenceTables.tsx`.

**[R06] API et infrastructure transversale :** `app/main.py`, `app/api/router.py`, `app/core/config.py`, `app/core/logging.py`, `app/core/exceptions.py`.

**[R07] Persistance et permissions communes :** `app/core/database.py`, `app/core/dependencies.py`, `app/core/enums.py`, `app/auth/permissions.py`, `app/*/models.py`.

**[R08] Services et acteurs :** `app/admin/`, `app/audit/`, `app/company/`, `app/investor/`, `app/researcher/`, `app/institution/`, `app/core/notifications.py`, `app/core/audit.py`.

**[R09] Authentification :** `app/auth/router.py`, `app/auth/tokens.py`, `app/auth/hashing.py`, `app/auth/revocation.py`.

**[R10] Protection des routes :** `app/auth/csrf.py`, `app/auth/rate_limit.py`, `app/core/redis.py`, `app/auth/permissions.py`, `docs/matrice-permissions.md` confronté aux routes.

**[R11] Fichiers :** `app/company/rapports.py`, `app/company/upload_validation.py`, `app/core/storage.py`, `app/auth/avatar.py`.

**[R12] Pipeline d'extraction :** `app/ingestion/docling_pipeline.py`, `app/ingestion/extractor.py`, `app/ingestion/semantic_search.py`. Le chemin OCR a aussi été vérifié dans les composants installés de Docling et RapidOCR.

## Références internes — suite

**[R13] Données et preuves :** `app/ingestion/schemas.py`, `app/ingestion/models.py`, `app/ingestion/proof_generator.py`.

**[R14] Scoring :** `app/scoring/engine.py`, `app/scoring/normalization.py`, `app/scoring/config_schema.py`, `app/scoring/models.py`, `config/weights/default.yaml`, `app/admin/review_queue.py`.

**[R15] Périmètres non implémentés :** `app/carbon/pcaf.py`, `app/carbon/emission_factors.py`, `app/carbon/models.py`, `app/explainability/decomposition.py`, `app/explainability/justification.py`, `app/ingestion/completeness.py`.

**[R16] Portefeuilles et change :** `app/investor/portfolio.py`, `app/investor/fx.py`, `app/investor/models.py`, `config/fx/rates.yaml`.

**[R17] Migrations :** `alembic.ini`, `alembic/env.py`, `alembic/versions/`, dont les révisions `b5f953dddf56`, `d00548652232` et `82f1c49778a5`.

**[R18] Déploiement :** `Dockerfile`, `docker-compose.yml`, `.env.example`, `.gitignore`, `scripts/dev-up.sh`, `README.md`.

**[R19] CI et tests backend :** `.github/workflows/ci.yml`, `tests/conftest.py`, `tests/integration/conftest.py`, `tests/unit/`, `tests/integration/`.

**[R20] Contrat d'API :** `scripts/export_openapi.py`, `frontend/orval.config.ts`, `frontend/src/shared/api/generated/`, scripts `api:generate` et `api:check` de `frontend/package.json`.

**[R21] Tests frontend :** `frontend/vite.config.ts`, `frontend/src/test/setup.ts`, `frontend/src/features/auth/components/LoginPage.test.tsx`, `frontend/src/shared/RequireRole.test.tsx`, `frontend/biome.json`.

**[R22] Supports et corpus :** `scripts/generate_reference_e2e_reports.py`, `data_test/reference_e2e/README.md`, `scripts/generate_final_presentation.py`, `docs/GreenFinance_Scorer_Soutenance_FINAL_REVISEE.pptx`.

**[R23] Diagramme :** `docs/uml/diagramme-cas-utilisation-reporting-esg.drawio`.

**[R24] Documentation et dépôt :** `README.md`, `ARCHITECTURE.md`, `FRONTEND-ARCHITECTURE.md`, configuration du remote GitHub et historique Git local. Les affirmations de ces documents ont été vérifiées contre le code actuel.